Indexing Near-Sorted Data

Aneesh Raman
aneeshr@bu.edu

Subhadeep Sarkar
ssarkar1@bu.edu

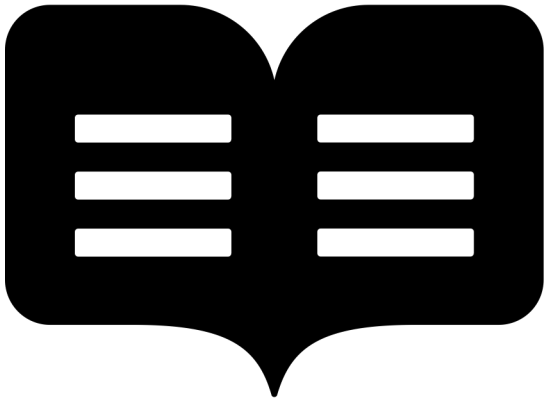
Matthaios Olma
maolma@microsoft.com

Manos Athanassoulis
mathan@bu.edu

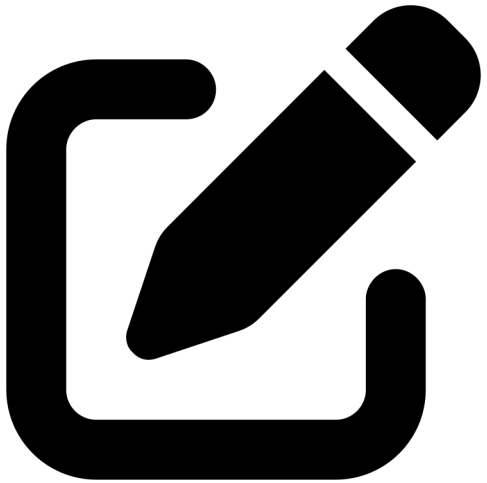
BOSTON UNIVERSITY

DiSC lab

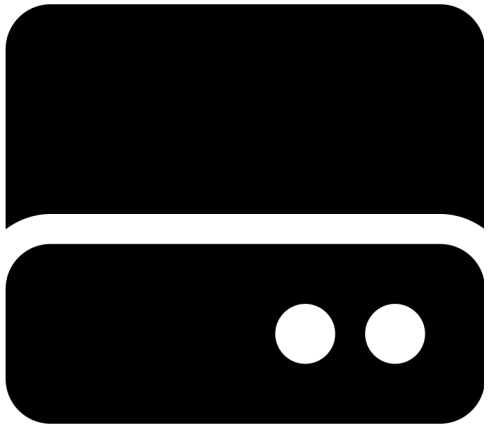
Indexes in Databases



Efficient queries

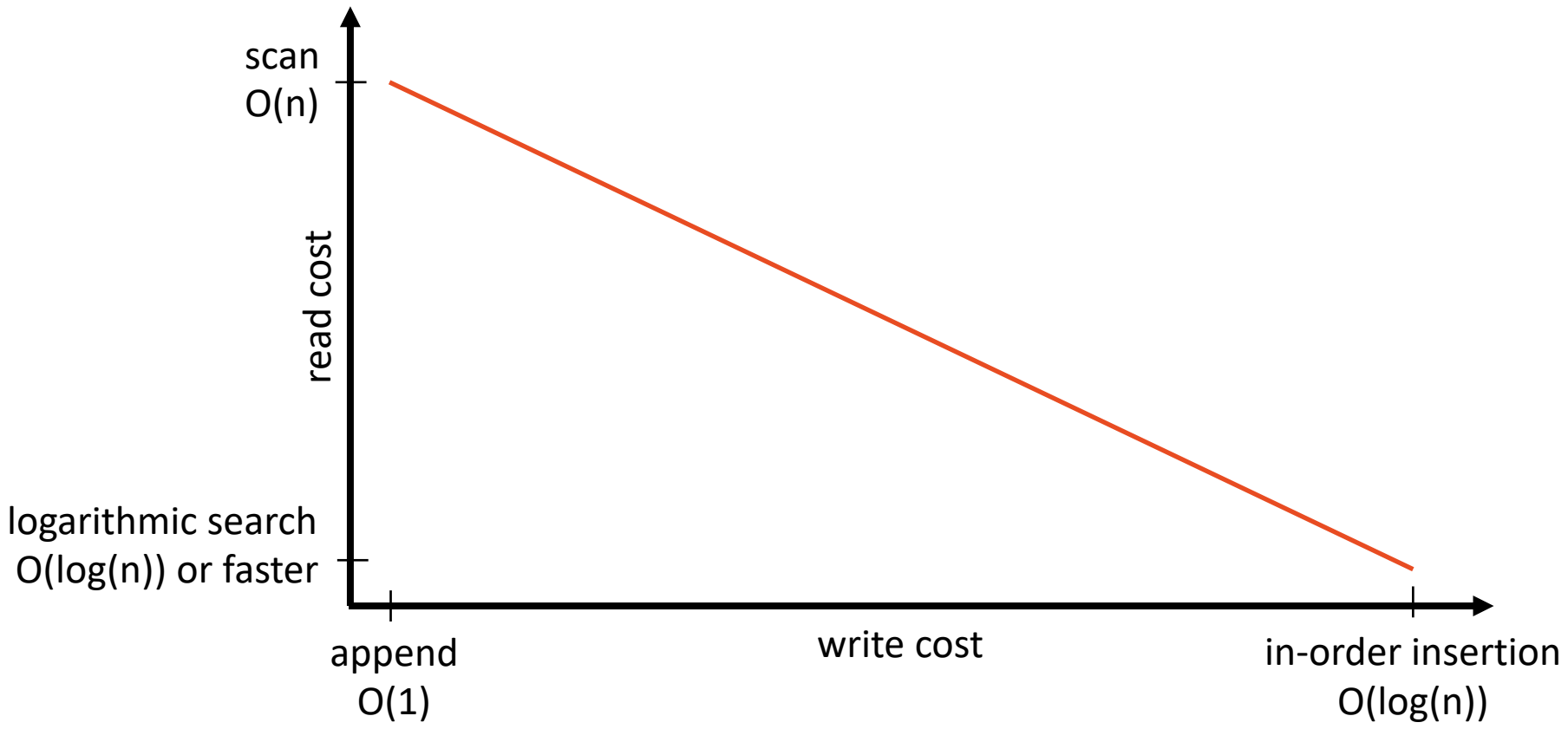


write amplification



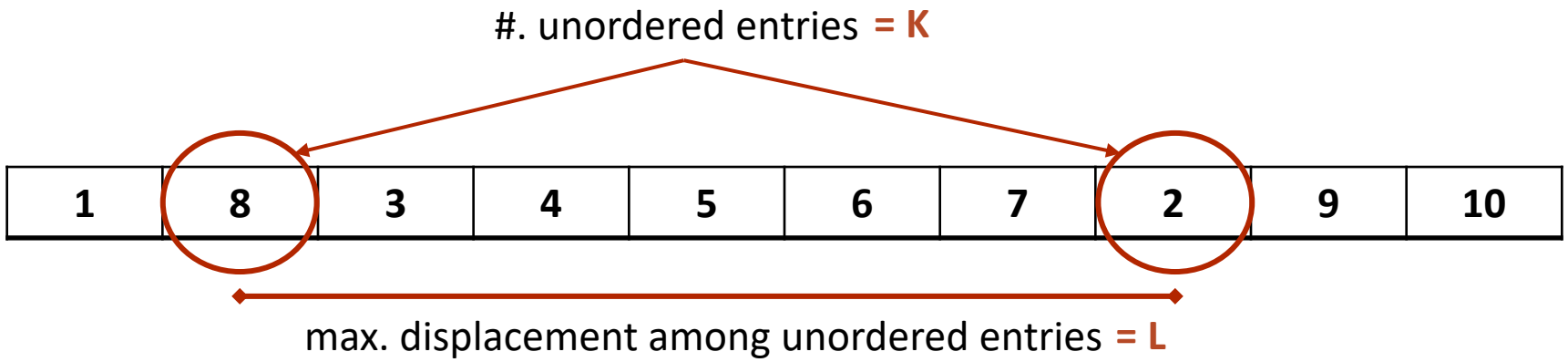
space amplification

Read-Write Tradeoff

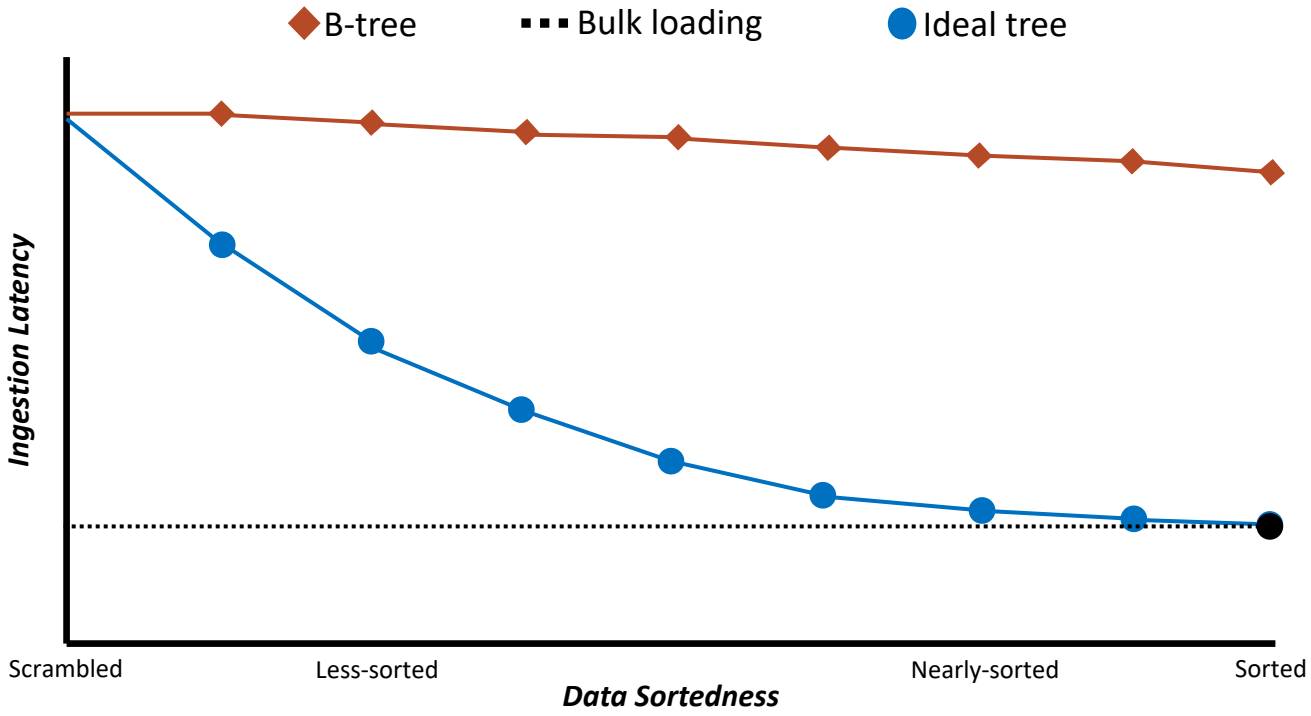


Data Sortedness

- #. Inversions, length of sub-sorted sequences, etc.
- Focus on a single metric to bring a data stream into sorted order
- K-L metric by Ben-Moshe (ICDT 2011)

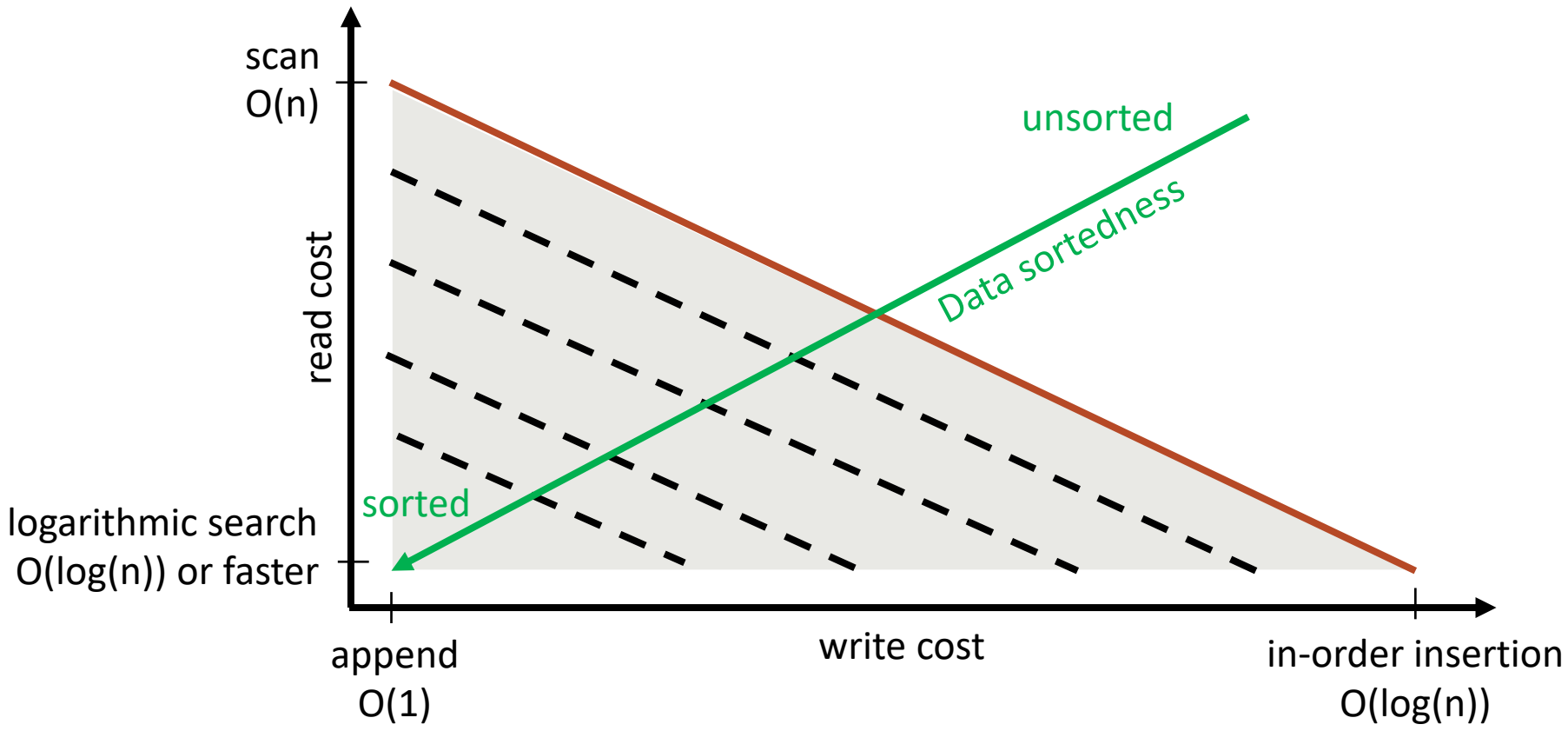


Problem


B-tree
Bulk loading
Ideal tree
Ingestion Latency
Scrambled
Less-sorted
Nearly-sorted
Sorted
Data Sortedness

Our Vision

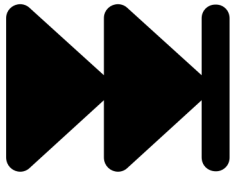
Our Vision


scan
O(n)
read cost
logarithmic search
O(log(n)) or faster
unsorted
Data sortedness
sorted
append
O(1)
write cost
in-order insertion
O(log(n))

Contributions

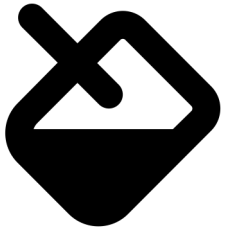
1. *"Sortedness"* as a resource
2. Index meta-design
3. *SWARE* design paradigm
4. Sortedness-aware Index (*SA* B^+-tree)

SWARE Design Elements



Rightmost leaf-appends

Bulk loading



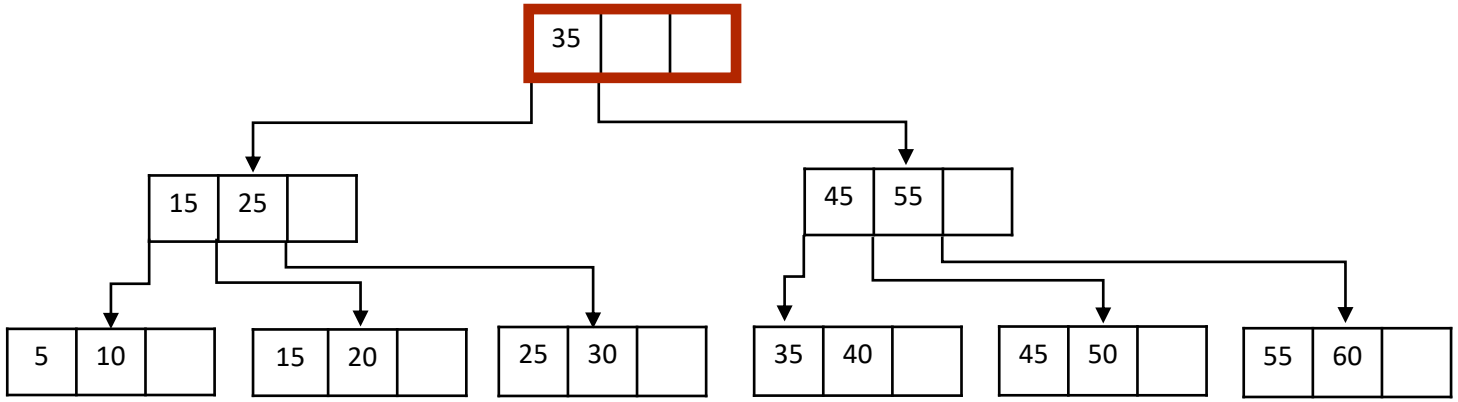
Variable fill & split factor

Buffering

Right-most Leaf Appends

Normal Insertion (top-insert)

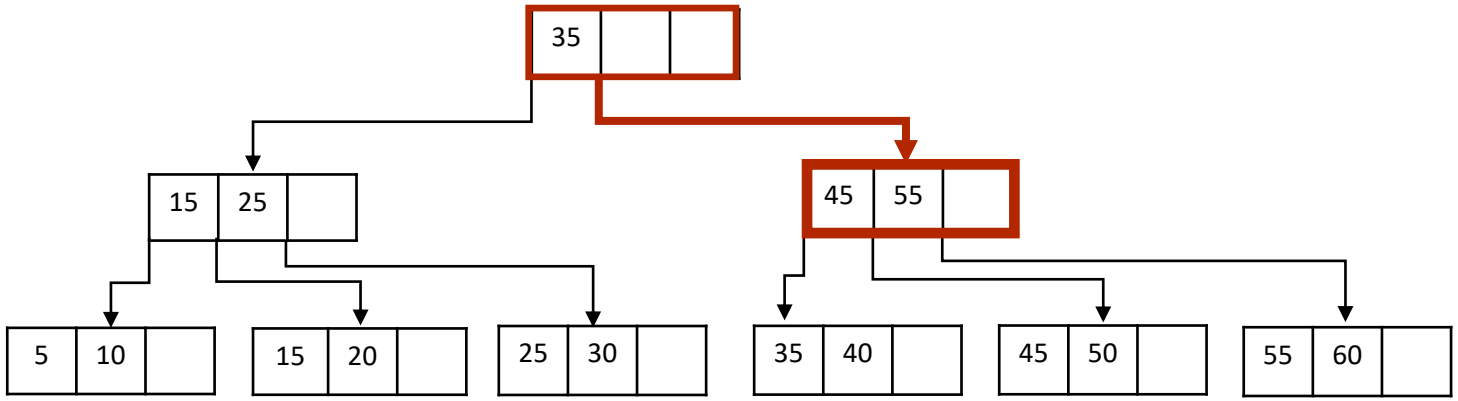
Insert Key 65



Right-most Leaf Appends

Normal Insertion (top-insert)

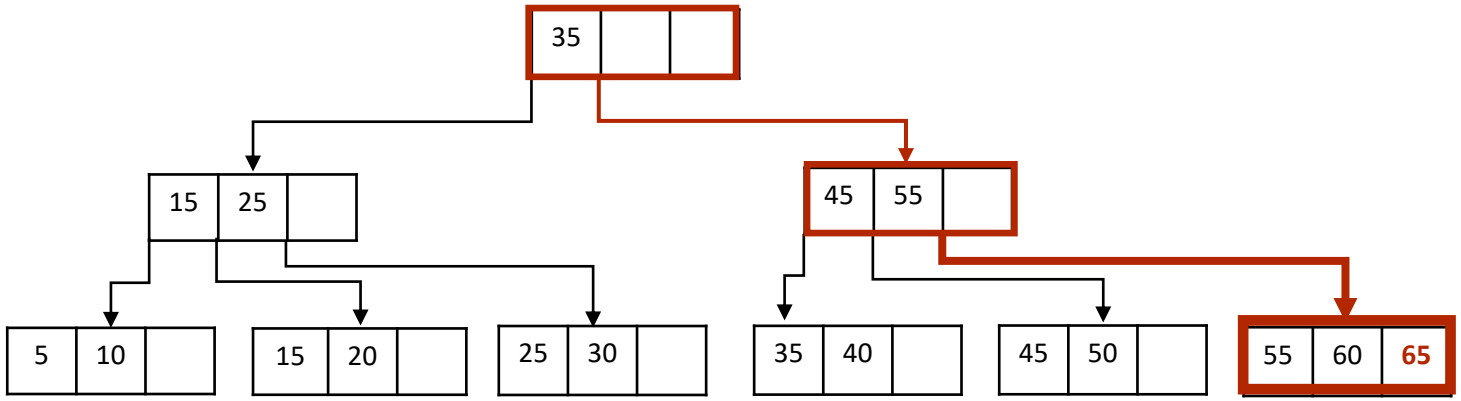
Insert Key 65



Right-most Leaf Appends

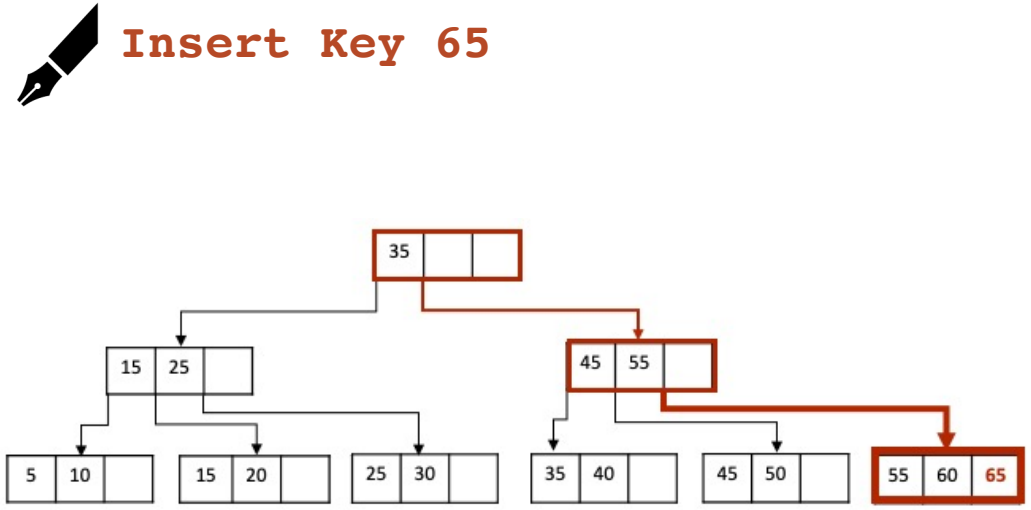
Normal Insertion (top-insert)

Insert Key 65

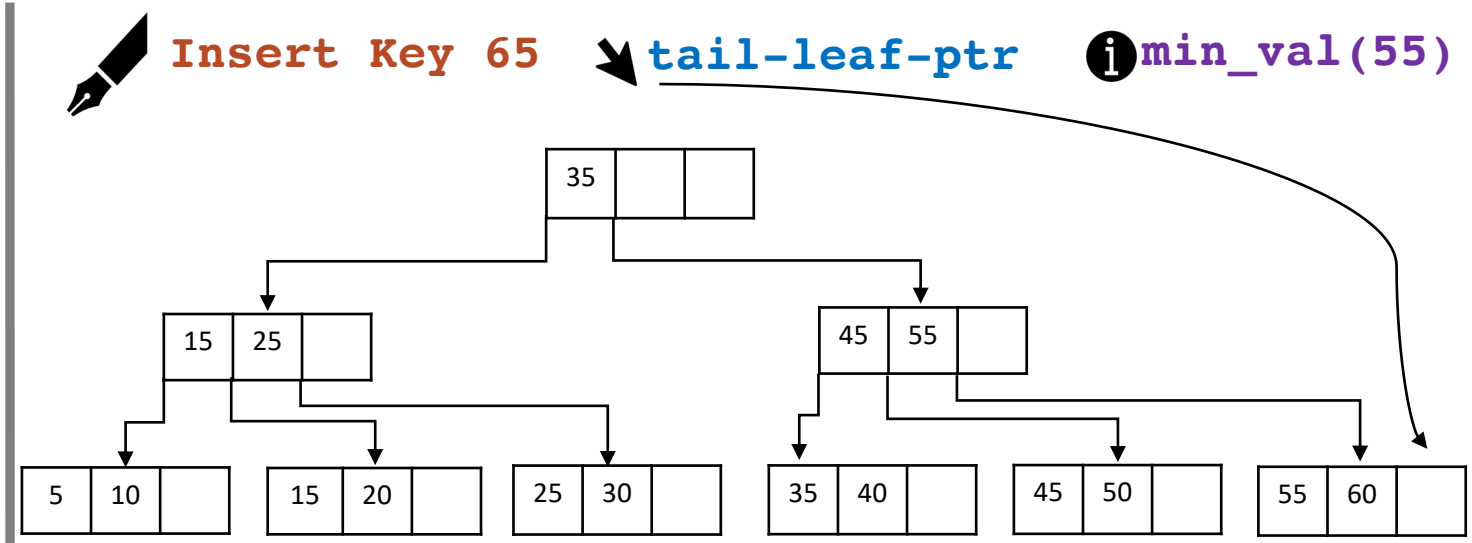


Right-most Leaf Appends

Normal Insertion (top-insert)

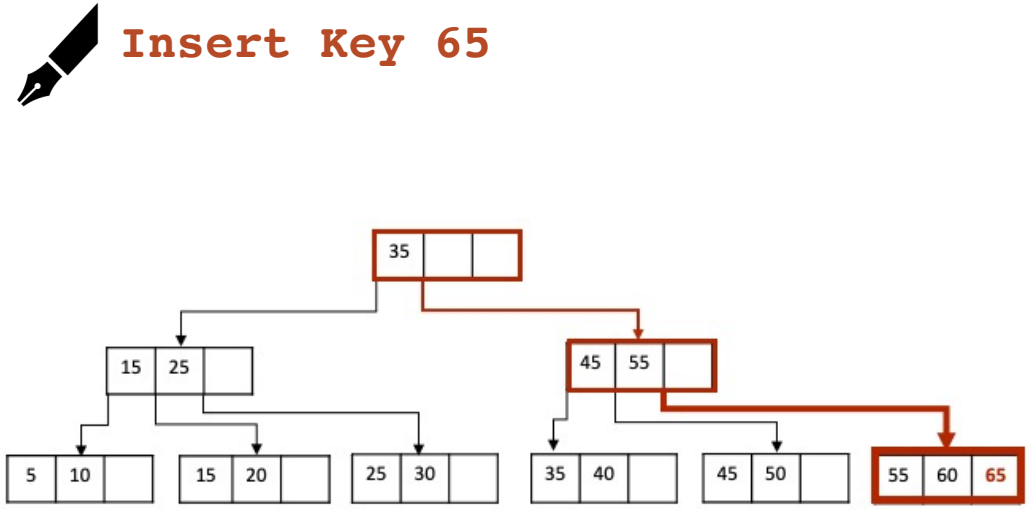


Insert to tail leaf

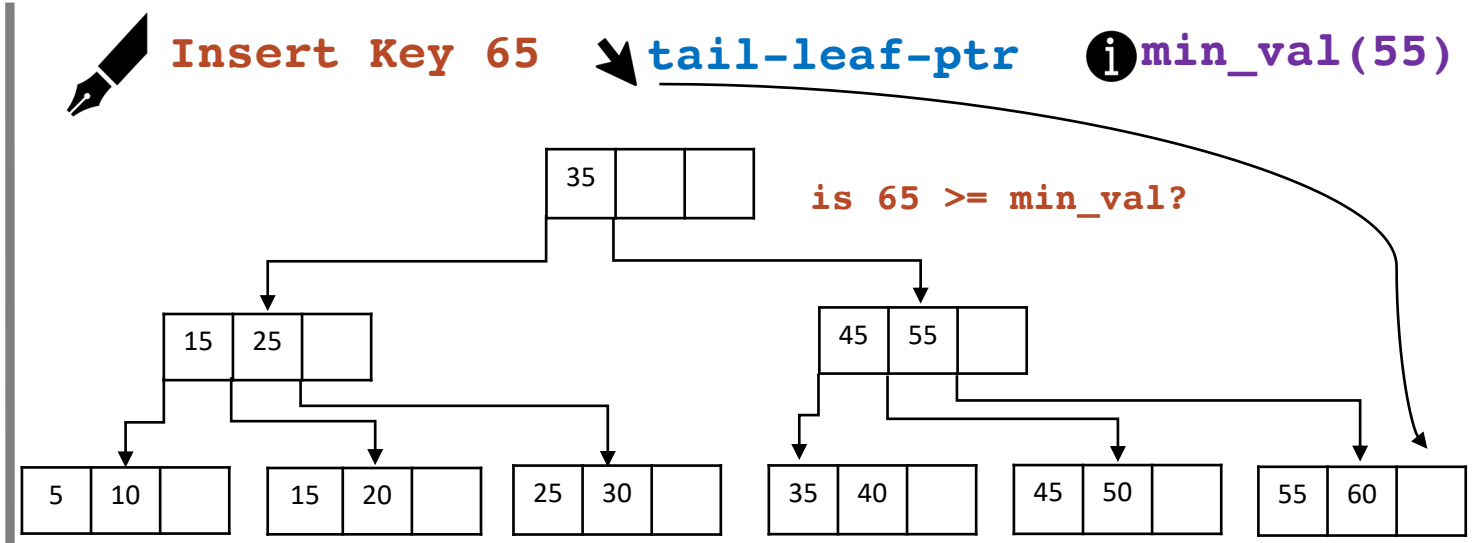


Right-most Leaf Appends

Normal Insertion (top-insert)



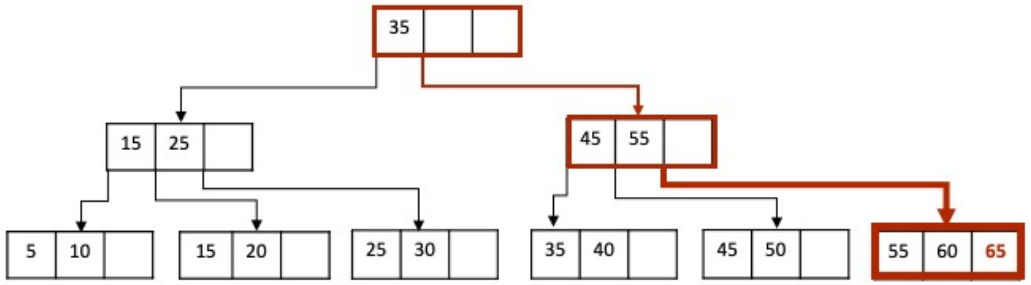
Insert to tail leaf



Right-most Leaf Appends

Normal Insertion (top-insert)

Insert Key 65

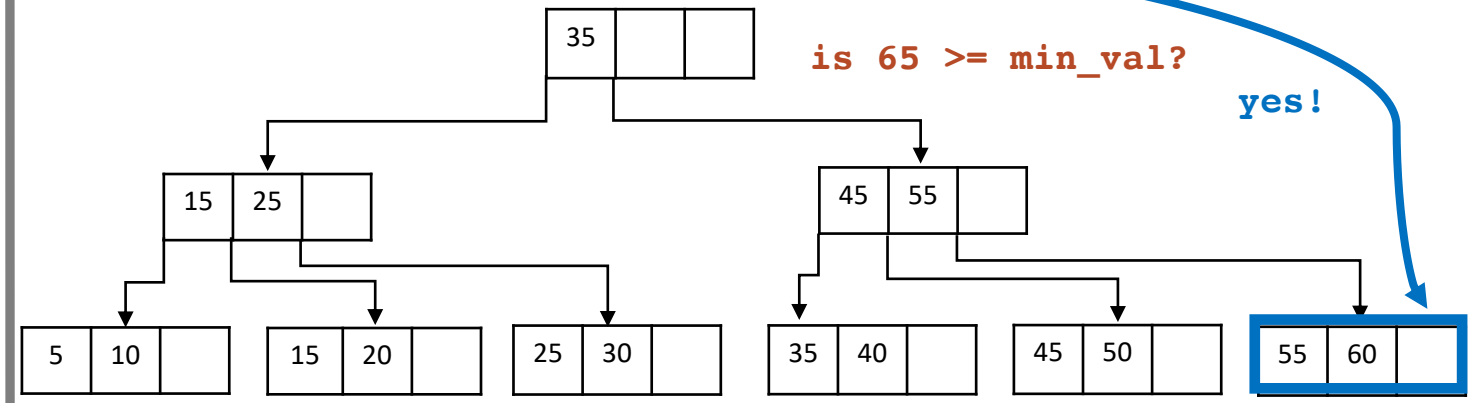


Insert to tail leaf

Insert Key 65

tail-leaf-ptr

min_val(55)



is 65 >= min_val?

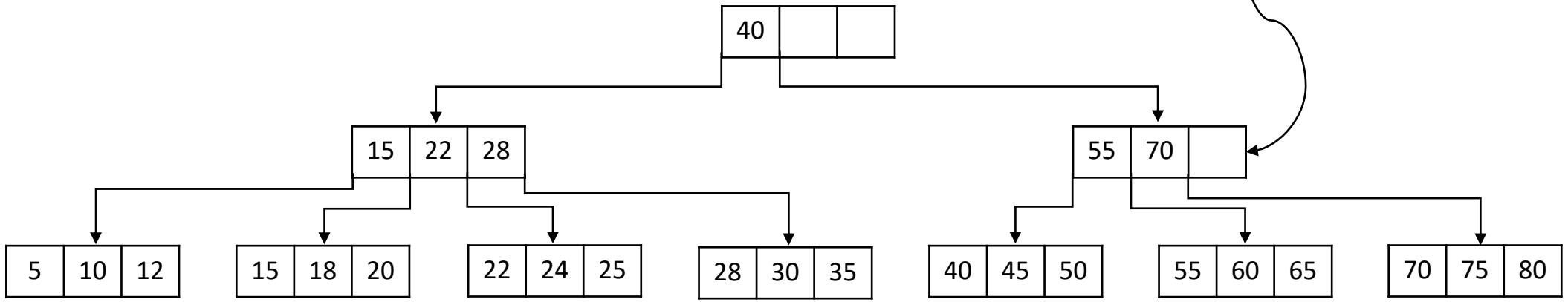
yes!

add key to tail leaf directly!

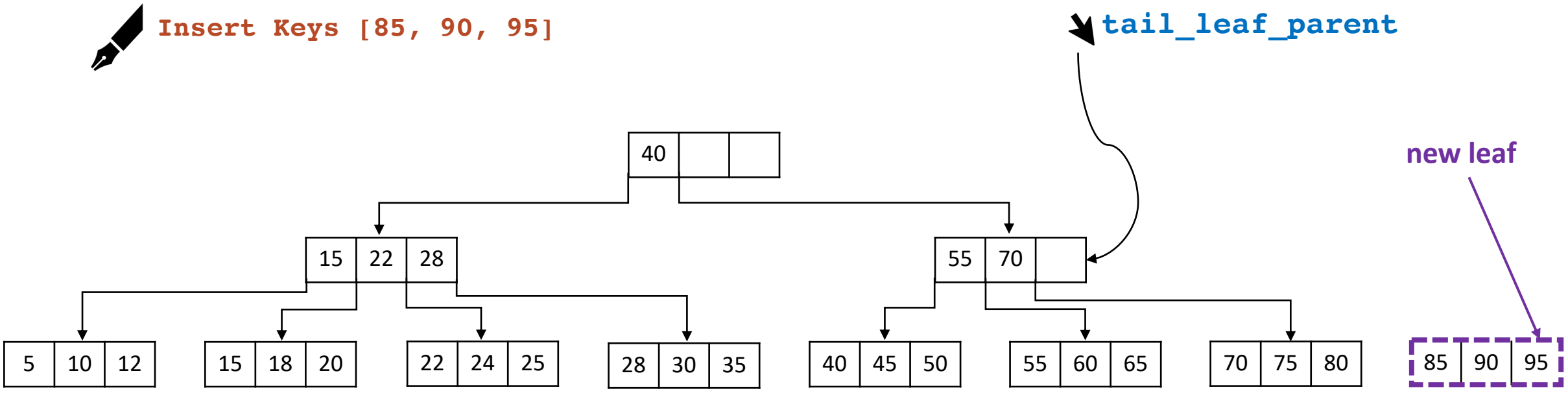
Bulk loading

Insert Keys [85, 90, 95]

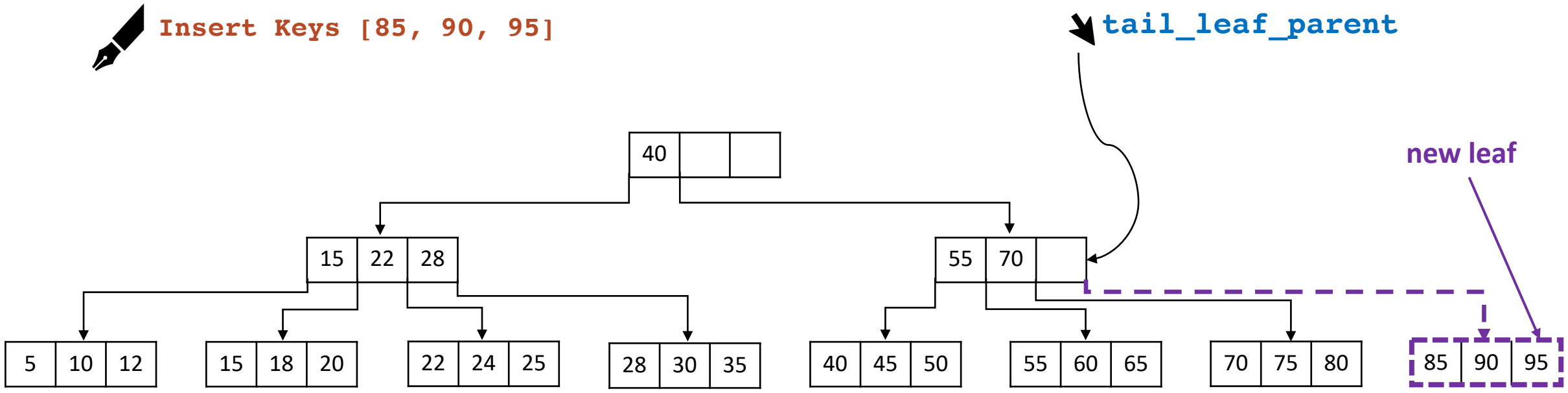
tail_leaf_parent



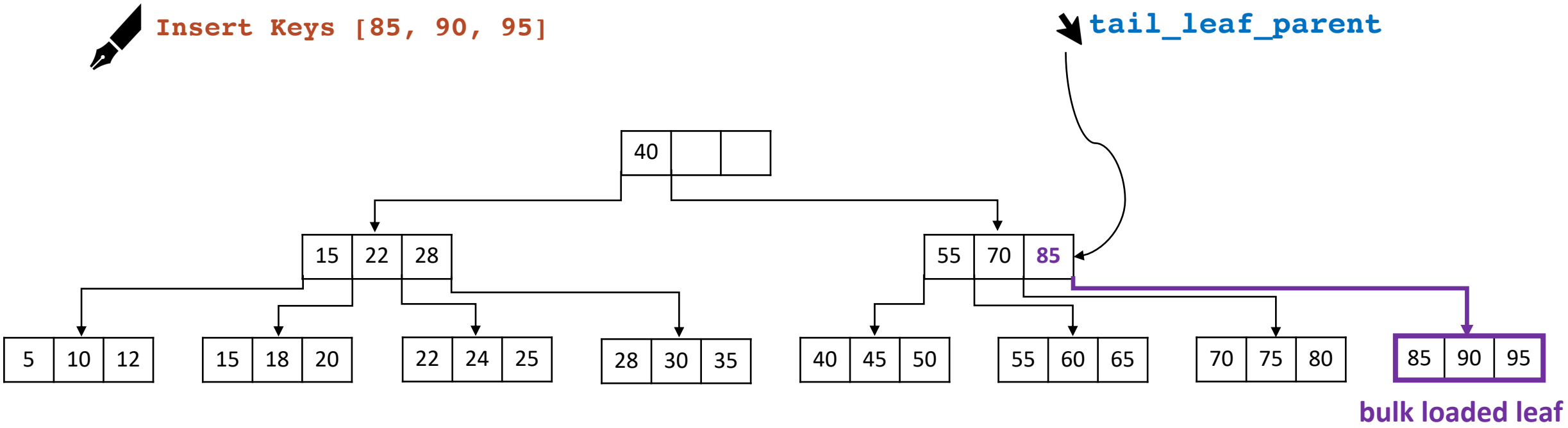
Bulk loading



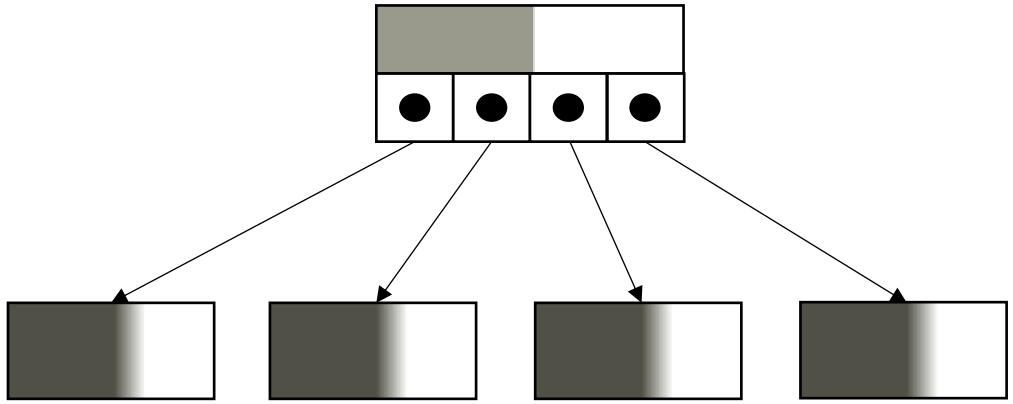
Bulk loading



Bulk loading

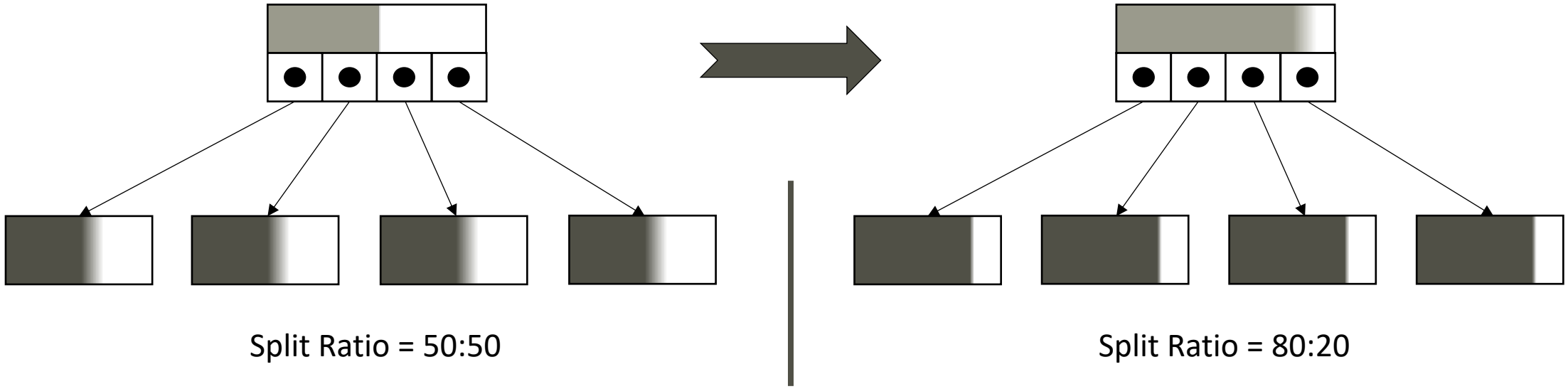


Adjusting Node Fill/Split Factor



Split Ratio = 50:50

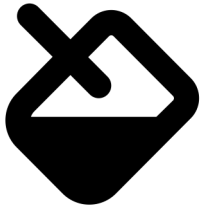
Adjusting Node Fill/Split Factor



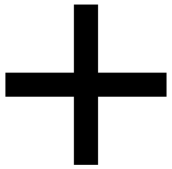
SWARE Ingestion

Rightmost leaf-appends

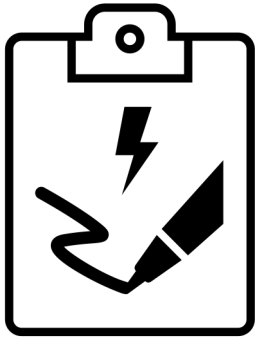
Opportunistic
Bulk loading



Variable fill
& split factor



Smart buffering



Faster ingestion

SWARE Ingestion

40	42	44	50	60	52	54	70

Rightmost leaf-appends

Opportunistic
Bulk loading

Variable fill
& split factor

+

Smart buffering

=

Faster ingestion

SWARE Ingestion

meta-data to identify sortedness

out of order entries

40	42	44	50	60	52	54	70

sorted entries

Rightmost leaf-appends

Opportunistic
Bulk loading

Variable fill
& split factor

+

Smart buffering

=

Faster ingestion

SWARE Ingestion

40	42	44	50	60	52	54	70

Opportunistically bulk load

Rightmost leaf-appends

Opportunistic
Bulk loading

Variable fill
& split factor

+

Smart buffering

=

Faster ingestion

SWARE Ingestion

60	52	54	70				

move remaining entries

Rightmost leaf-appends

Opportunistic
Bulk loading

Variable fill
& split factor

+

Smart buffering

=

Faster ingestion

SWARE Ingestion

52	54	60	70				

sort remaining entries

Rightmost leaf-appends

Opportunistic
Bulk loading

Variable fill
& split factor

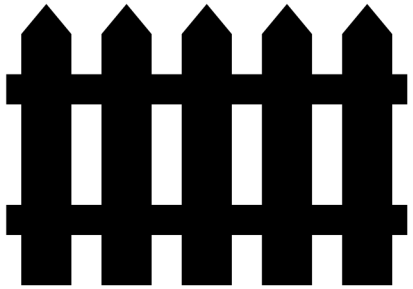
+

Smart buffering

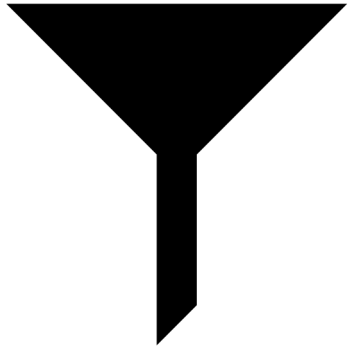
=

Faster ingestion

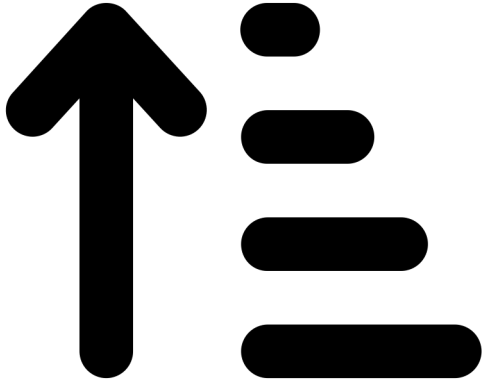
Optimizing Reads



zonemaps

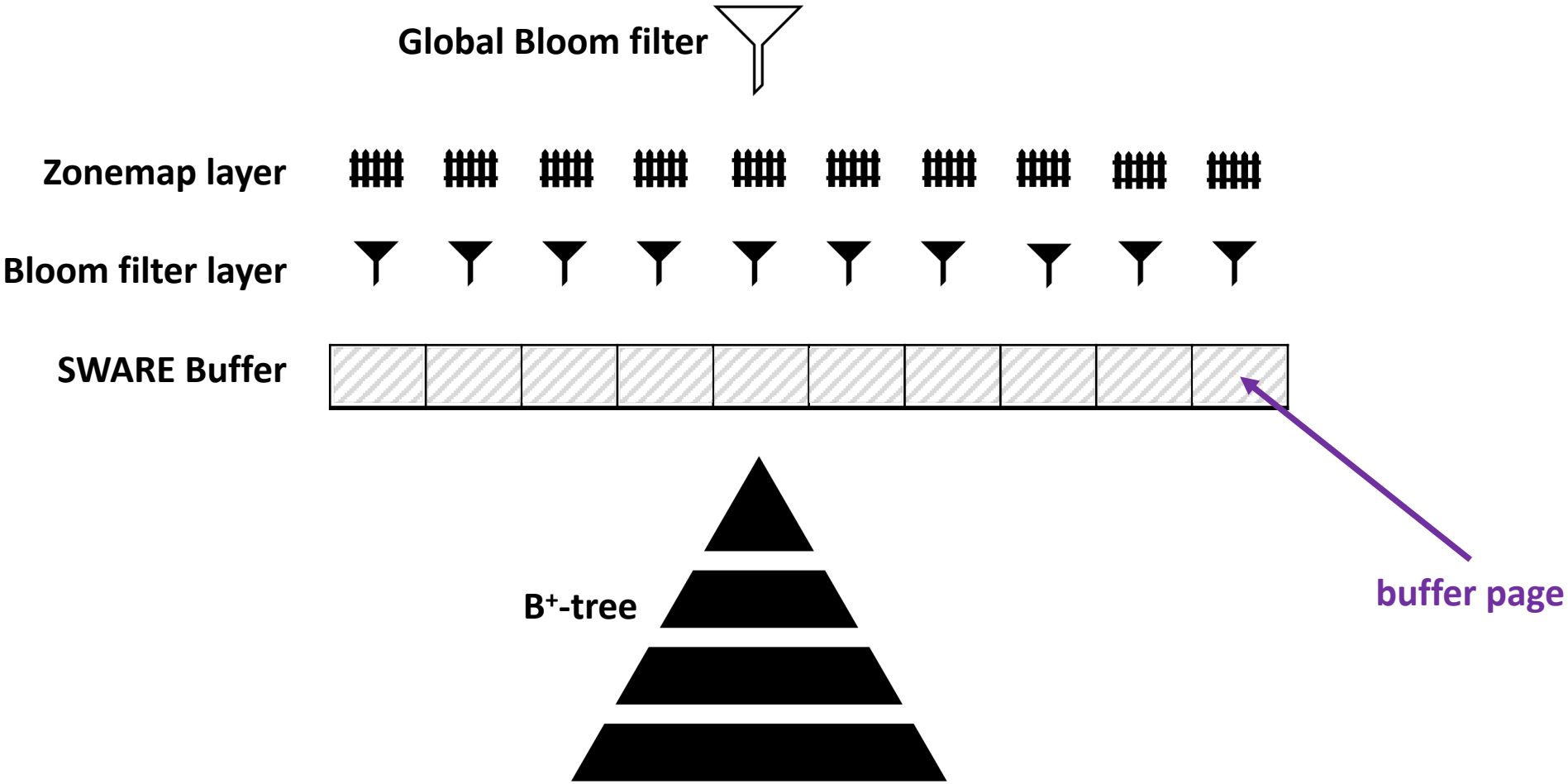


Bloom filters

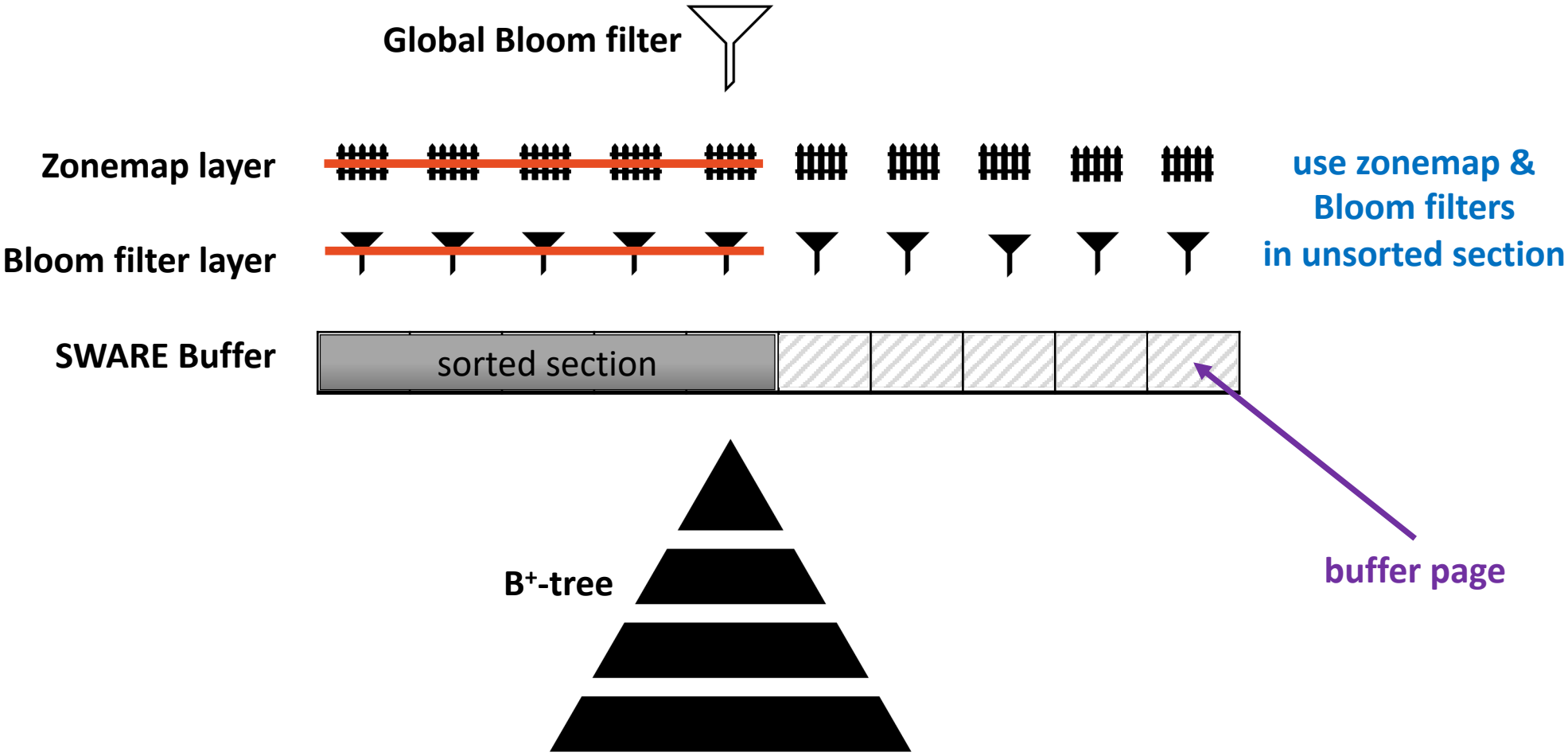


query-driven
partial sorting

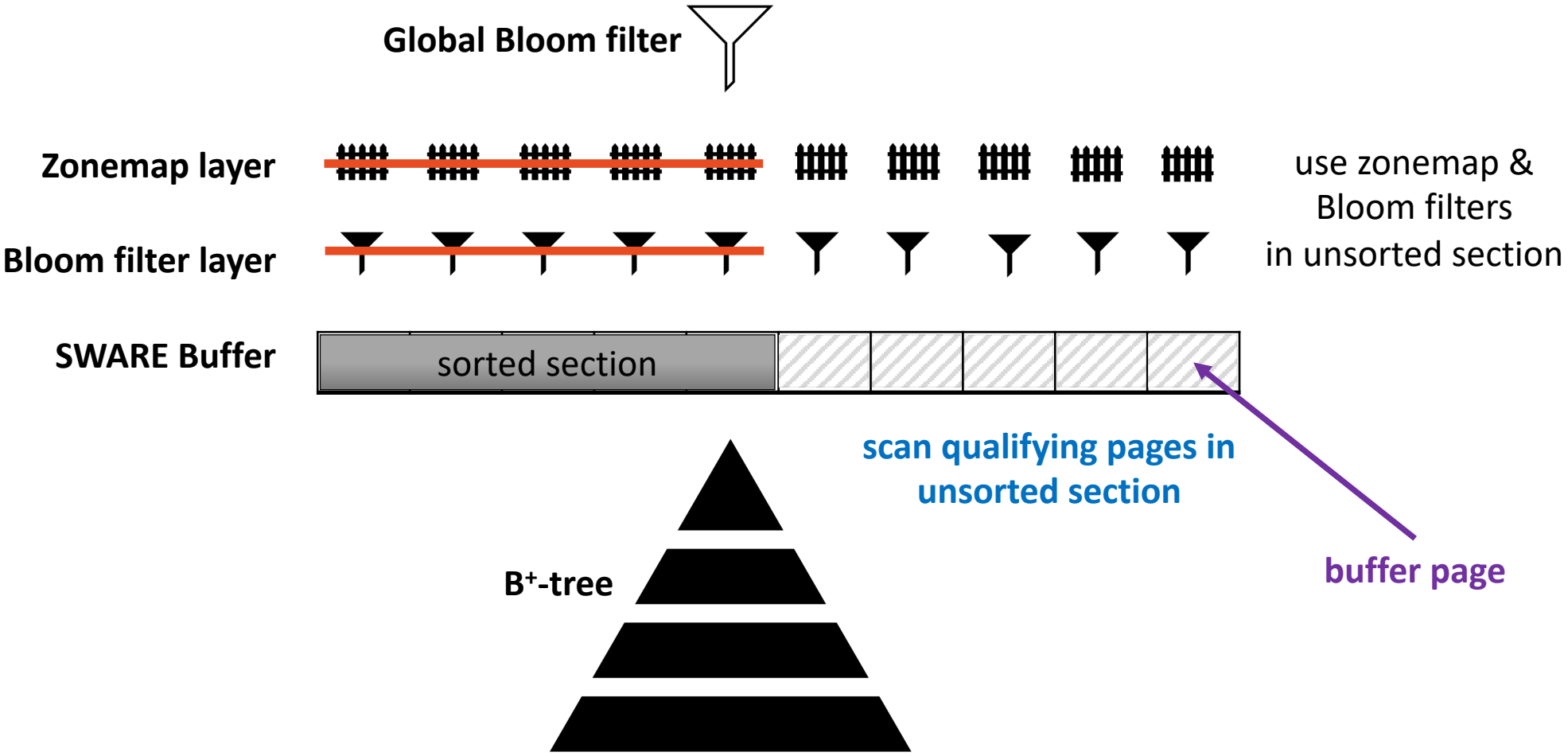
Overall Structure



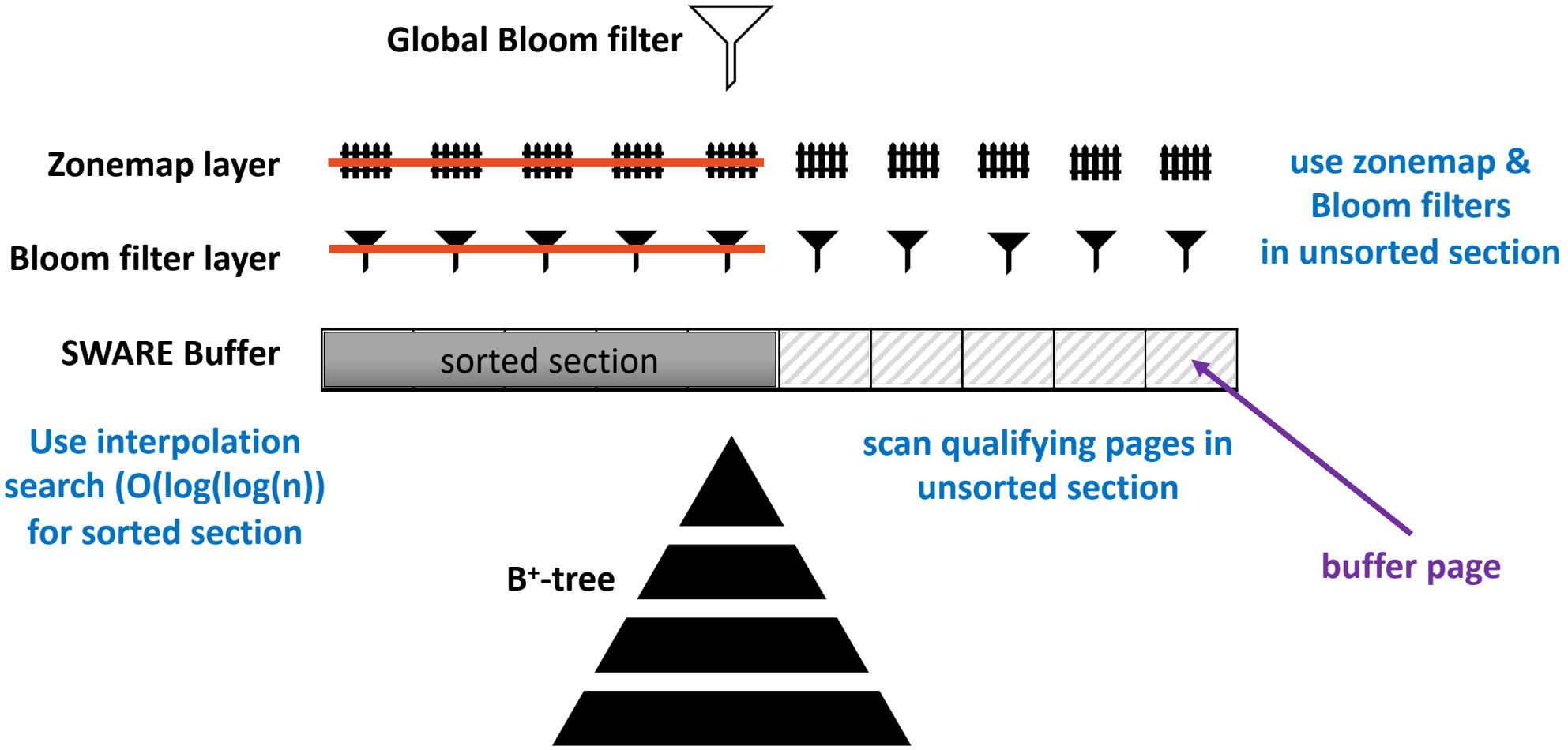
Overall Structure



Overall Structure



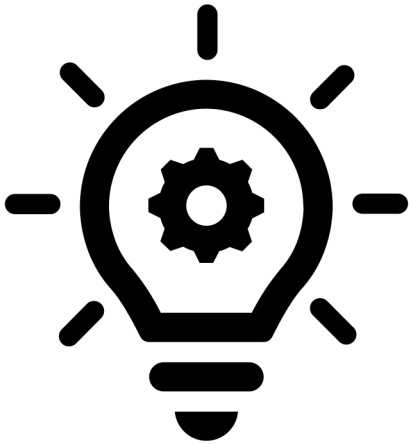
Overall Structure



Scanning Unsorted Section

unsorted section scan is still expensive!

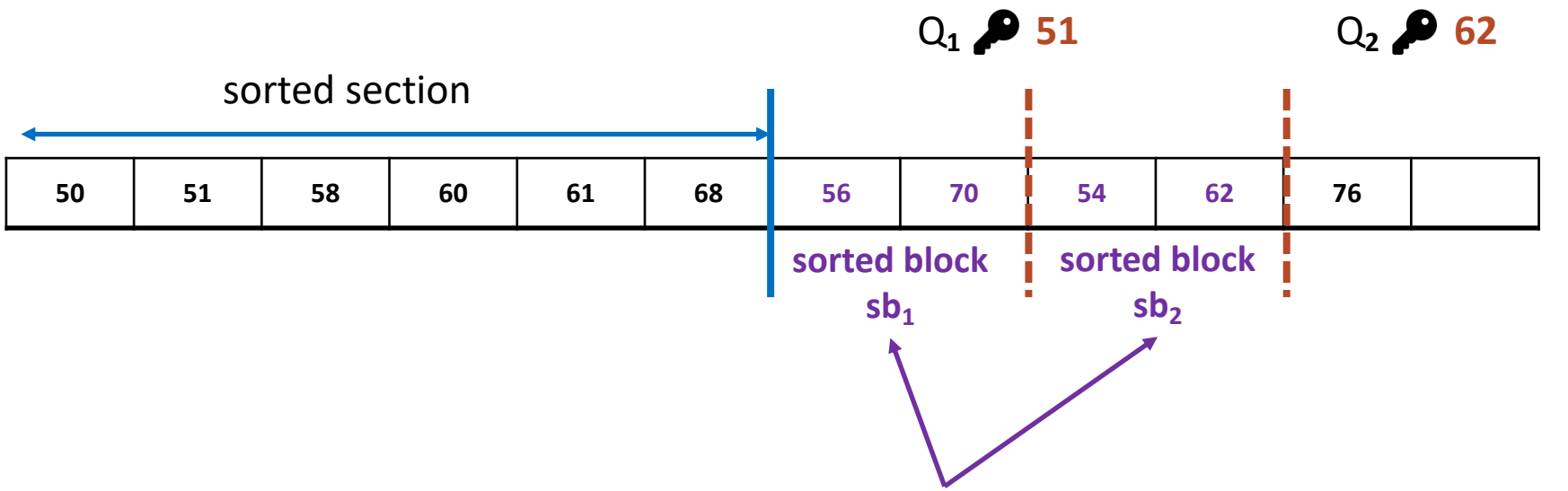
Scanning Unsorted Section



increase sorted component

Query-driven partial sorting

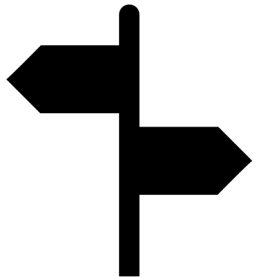
Query-driven Partial Sorting



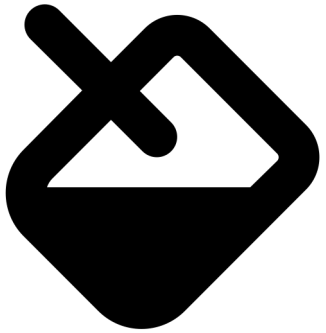
use faster search algorithm (e.g., interpolation search / binary search)

simply merge sorted components => also helps speed up inserts!

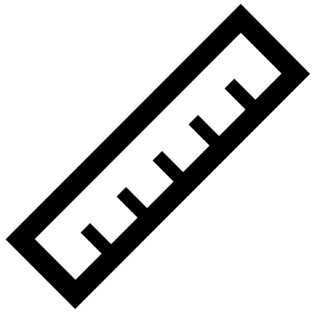
Tuning Decisions



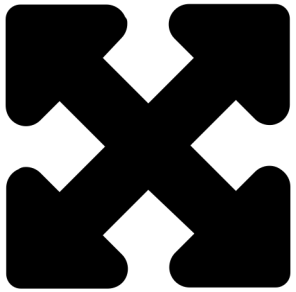
sorting algorithm



fill & split factor



buffer size



query sorting threshold

Experimental Setup

Metrics:
1. Overall performance (speedup)
2. Raw performance (latency)

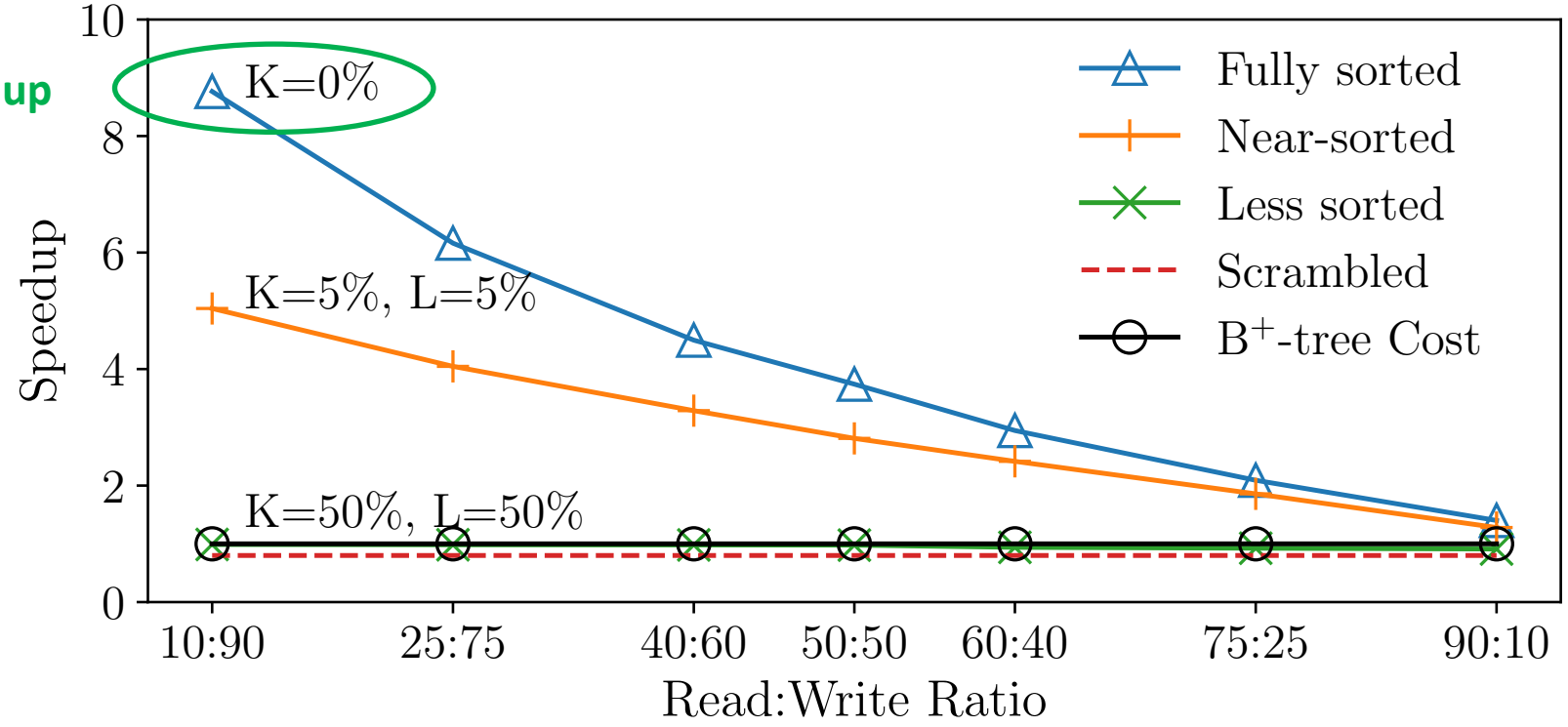
Workload Generator:
1. 500M Integer keys (~ 4GB)
2. Random lookups on existing keys

System Setup:
1. Intel Xeon Gold 5230
2. 2.1GHZ processor w. 20 cores
3. 384GB RAM, 28MB L3 cache

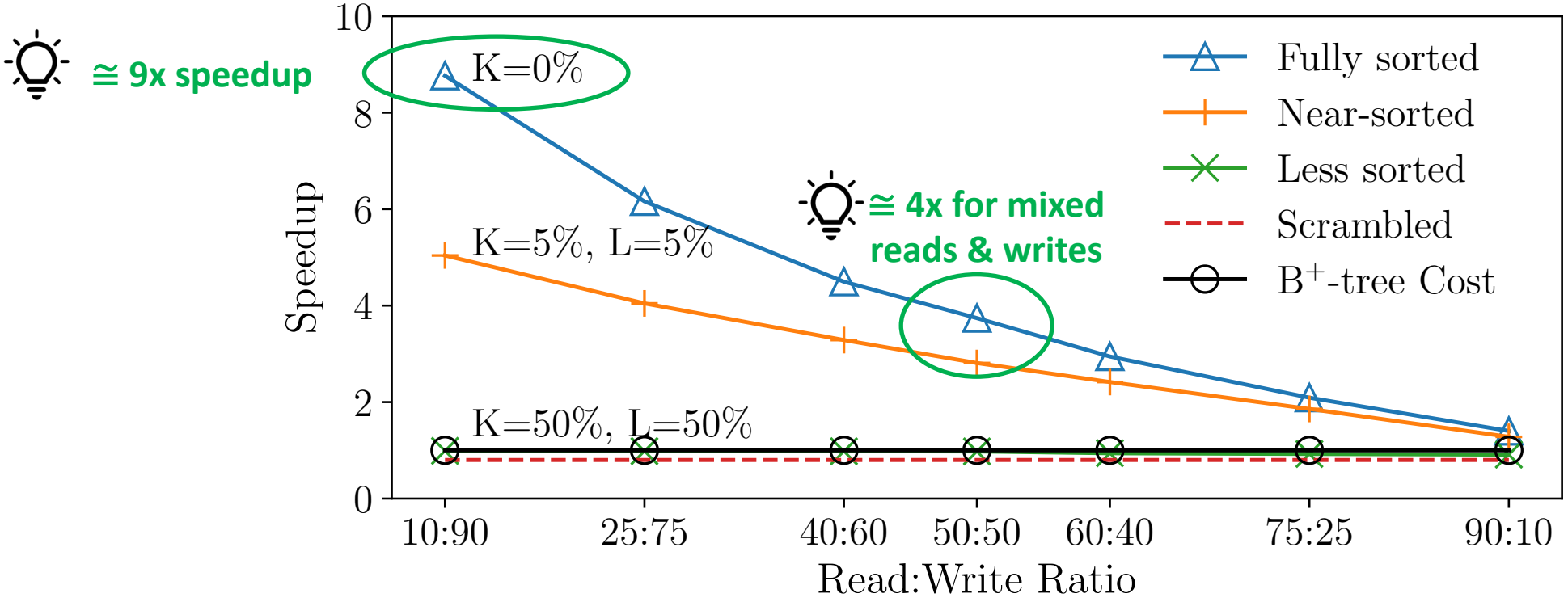
Default Index Setup:
1. Buffer = 40MB (5M); flush <= 50%
2. BFs = 10 BPK; Murmur Hash
3. Split = 80:20; Bulk load = 95%

Overall Performance

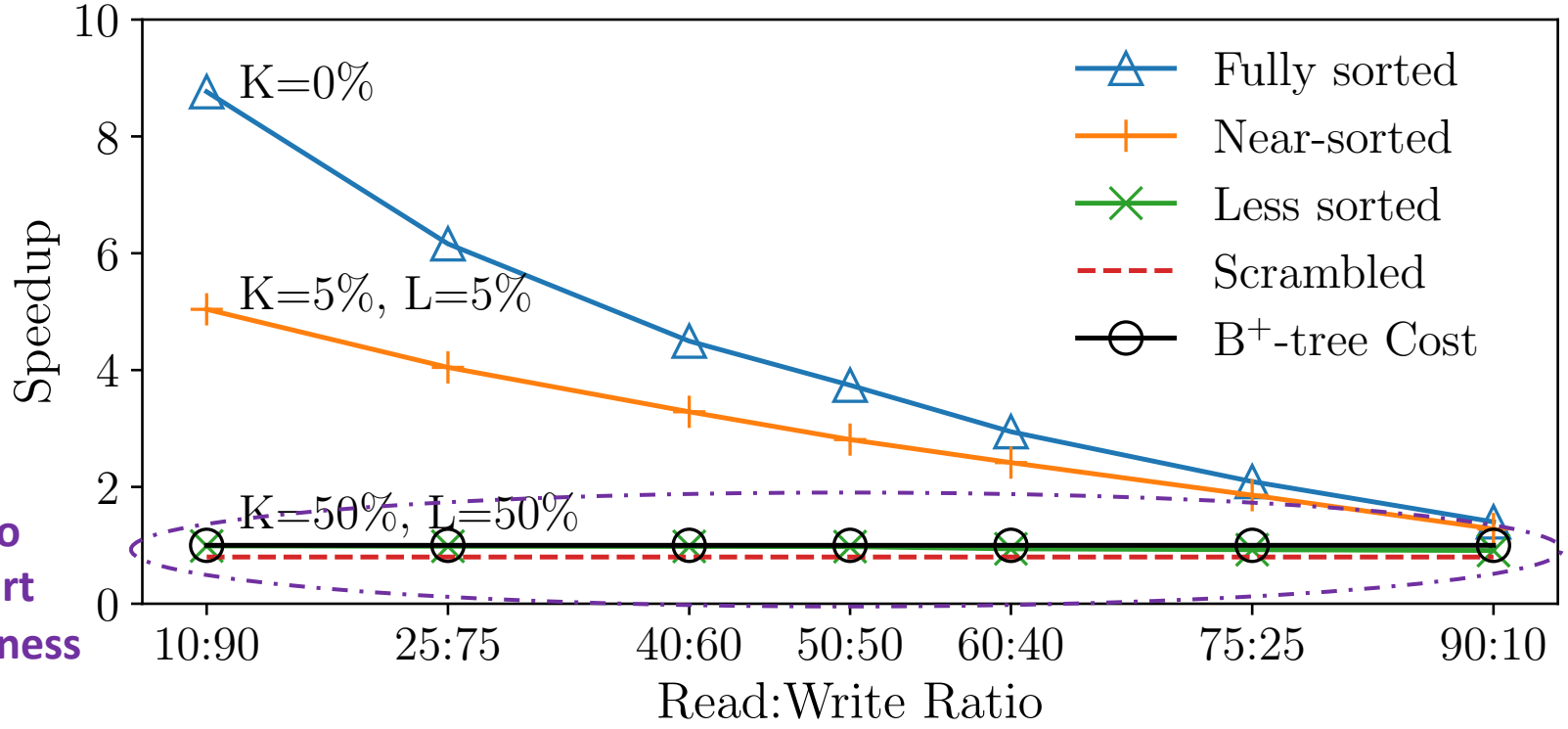
≅ 9x speedup



Overall Performance

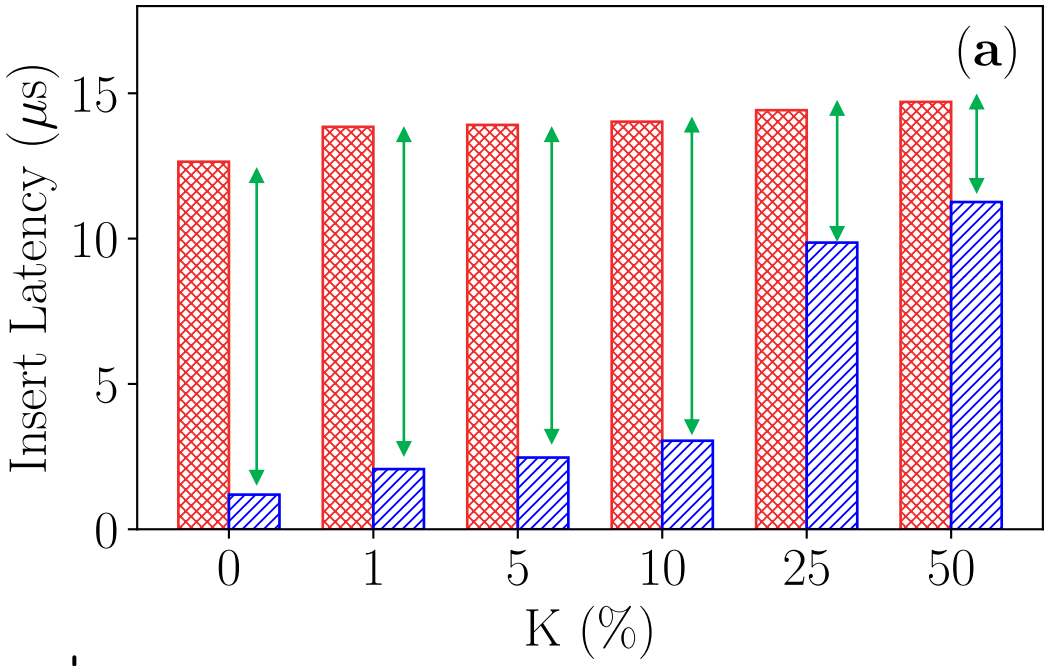


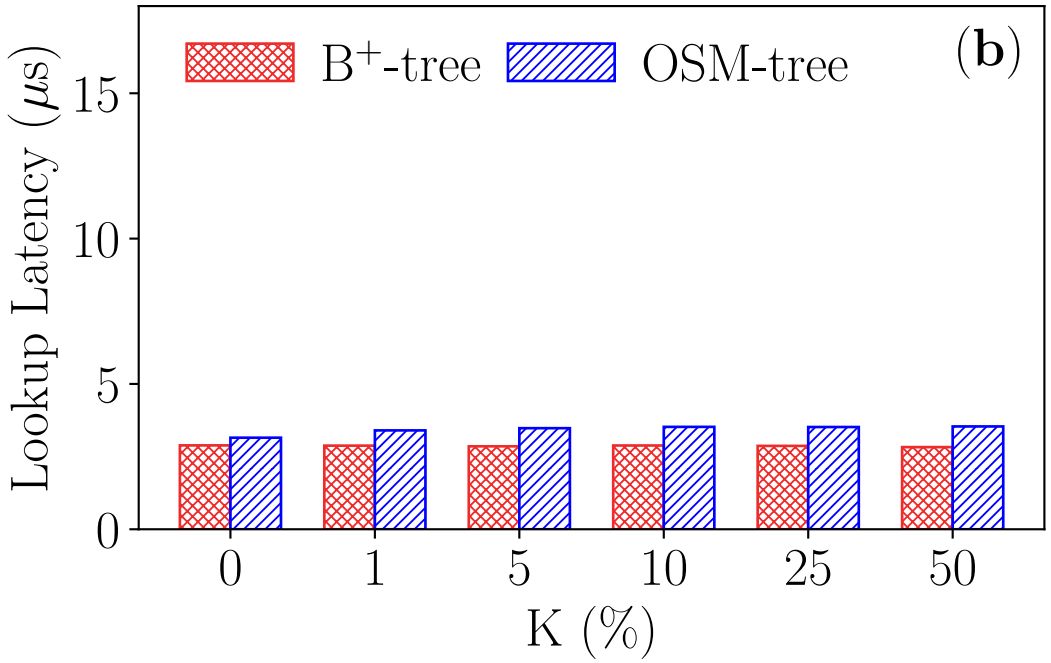
Overall Performance



comparable to state-of-the-art with less sortedness

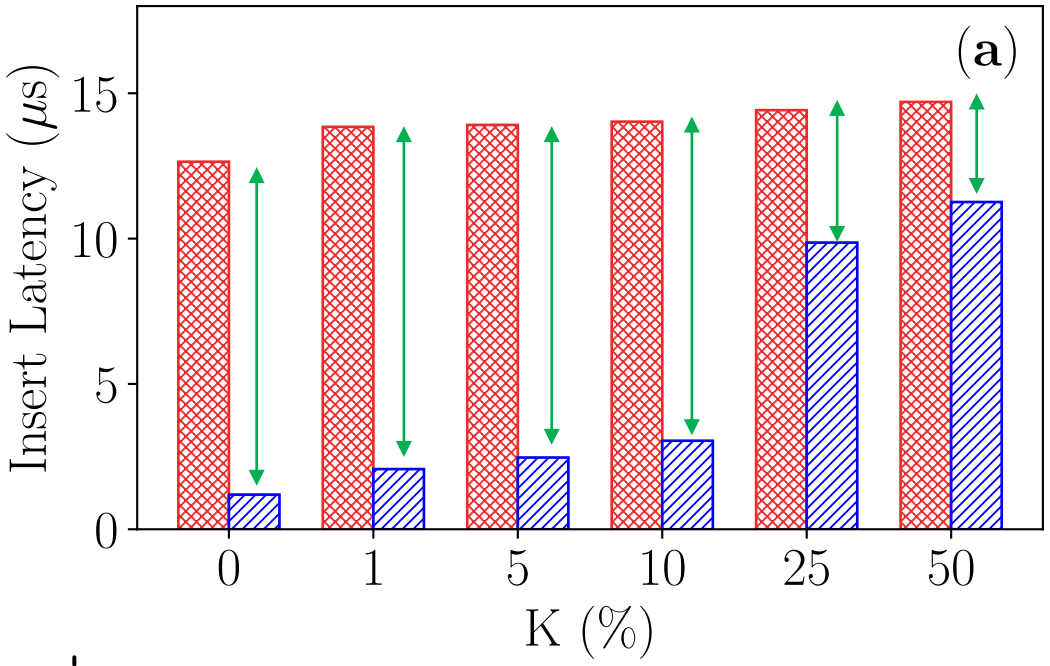
Raw Performance



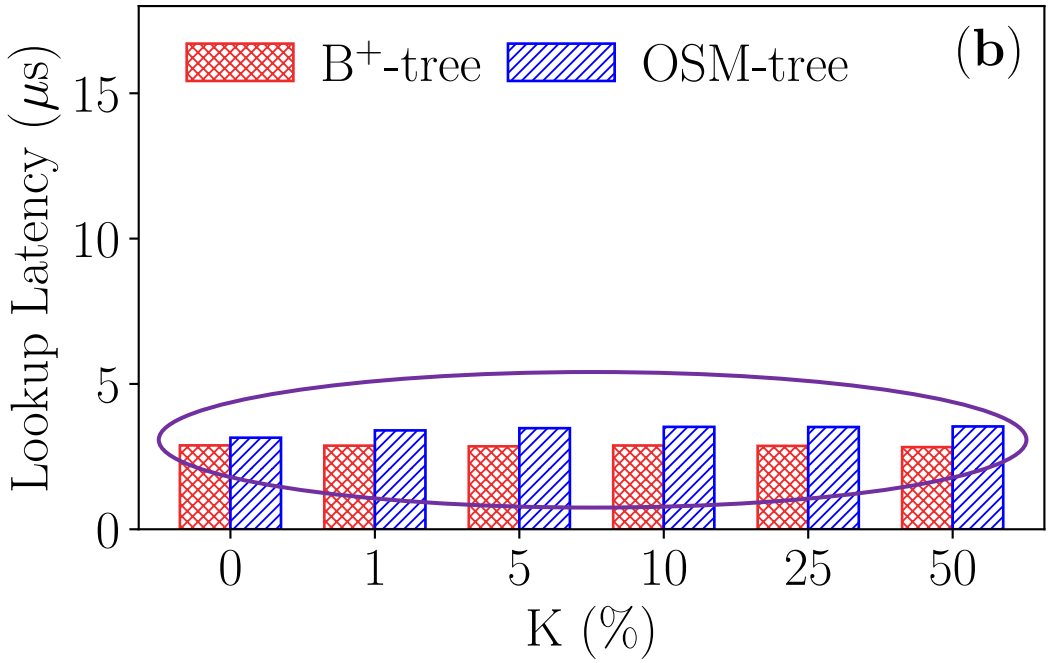


ingestion latency reduced between 27-90%

Raw Performance



ingestion latency reduced between 27-90%



overhead in lookups between 5-26%

Summary

Indexes can use "**sortedness**" as a resource

Smart buffering + bulk index appends = **faster** inserts

8.8x speedup with **SWARE** meta-design

Framework can be **extended** to other indexes

Thank You!

DiSC lab